Universidad de las Fuerzas Armadas ESPE

Departamento Ciencias exactas

NRC: 36486

Carrera Telecomunicaciones

Apellido y nombre. Gualotuña Gualotuña Erick Mateo.

Asignatura fundamentos de la programación

Problema 1.1 . Media de dos valores

1. Análisis del problema: Dos num

• Entrada : dos numeros . los llamaremos num 1 y num 2

• Proceso : Calcular la media aritmetica . la formula es. media = (num 1 + num 2) / 2

• Restricciones: Se asume que el usuario ingresara números validos . no se especifica si la media debe ser entero o decimal, asi que calculamos como un numero decimal (real) para mayor precisión.

• salidas: El valor de la media calculado

2. Tabla de objetos.

Objeto	Nombre	Valor	Tipo
Primer Numero	num 1	Ingresado	real
Segundo numero	num 2	Ingresado	real
Resultado de la media	media	calculado	real

3. Seudocodigo:

```
Algoritmo Media_Dos_Valores
// 1. Declaracion de variables
Definir num1, num2, media como Real
// 2. Entrada de datos
Escribir "Ingrese el primer numero"
Leer num1
Escribir "Ingrese el segundo numero"
Leer num2
// 3 paso (cálculo)
media <- (num1 + num2) / 2.
// 4. salida de resultados
Escribir "La media de los dos valores es: ", media
fin Algoritmo.
```

4. Diagrama de flujo



Problema 1.2 Valor absoluto de x al cubo

1. Análisis del problema.

Entrada: Un número real x. Por ejemplo, si el usuario ingresa -4

Proceso: 1. Calcular el valor absoluto de x. Para -4, el valor absoluto es 4.
2. Elevar ese valor absoluto al cubo. Para 4, sería 4·4·4 = 64.

Salida: El resultado final. Para este caso, 64.

Restricciones: El número puede ser positivo, negativo o cero. Usaremos variable de Tipo real para permitir decimales.

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Número ingresado	x	-4	real
Valor absoluto x	abs_x	4	real
Resultado final	resultado	64	real.

3. Seudocódigo.

```
Algoritmo ValorAbsolutoAlCubo

//1. Declaración variables
Definir x, abs_x, resultado como Real

//2. Entrada de datos
Escribir "Ingrese un número"
Leer x // "-4"

//3. Proceso
calcular el valor absoluto
si x < 0 entonces
    abs_x ← x * -1 // abs_x = -4 * -1 = 4
Sino
    abs_x ← x
finSi

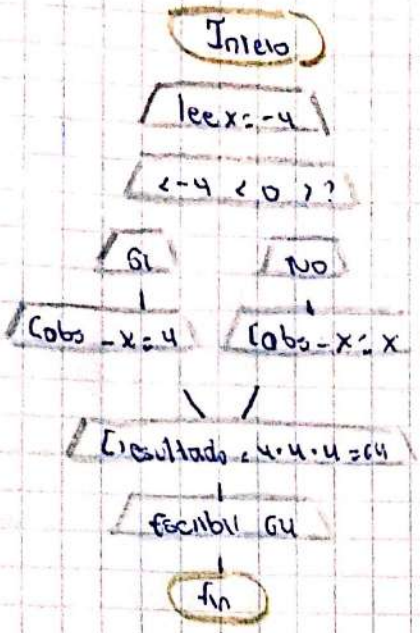
//elevamos al cubo
resultado = 4 * 4 * 4 = 64 //

//4. Salida de resultados
Escribir "El valor absoluto de ", x, " al cubo",
Salida "El valor absoluto de -4 al cubo es 64"

finAlgoritmo.
```

Escaneado con CamScanner

4 Diagrama de flujo



Problema 1.3 Divisible.

•Entrada: Dos números enteros. Llamaremos x al primero (dividendo) e y al segundo (divisor)

•Proceso: 1. Verificar que y no sea cero.
2. Calcular el resto de la división de x entre y usando el operador módulo. resto = x % y.
3. Si resto = 0, entonces x es divisible entre y. En caso contrario no lo es.

Salida: Un mensaje indicando si el primer número es divisible por el segundo o no

Restricciones: El divisor y debe ser diferente de cero. Si y=0 el programa debe mostrar un error

2. Tabla de valores

Objeto	Nombre	Valor	Tipo
Primer número	x	10	entero
Segundo número	y	2	entero
Resto de la división	resto	0	entero

3.5 Pseudocódigo.

Algoritmo Divisibles

//1. Declaración de variables
Definir x, y, resto como Entero

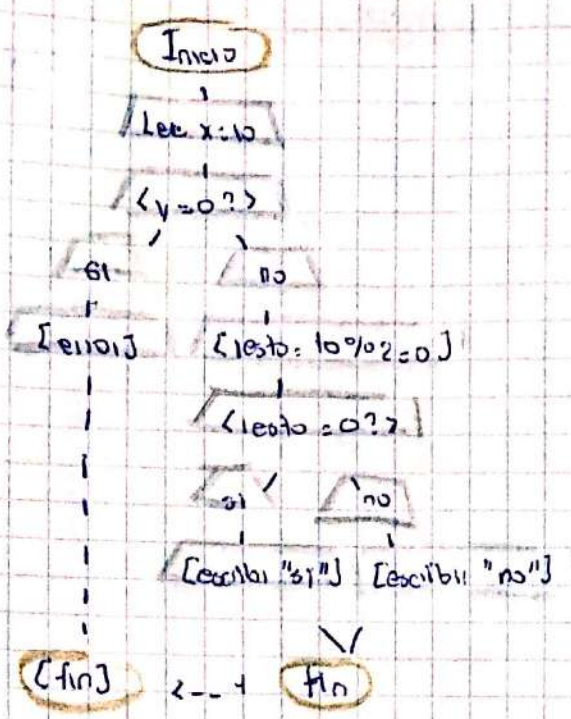
//2. Entrada datos
Escribir "Ingrese el primer número (dividendo): "
Leer x //10
Escribir "Ingrese el segundo número (divisor): "
Leer y //2

//3. Proceso y validación
Si y = 0 entonces
Escribir "Error: No se puede dividir entre cero"
Sino
//calcular el resto de la división"
resto <- x % y //resto = 10 % 2 = 0

//Verificar si el resto es cero
Si resto = 0 entonces
Escribir x, "Sí es divisible entre" y
Sino
Escribir x, "No es divisible entre" y
FinSi
FinAlgoritmo.

Escaneado con CamScanner

4. Diagrama de flujo



5. Prueba de escritorio

Caso	x	y	¿y=0?	resto = x %0 //	¿res=0?	Salida esperada
1	10	2	NO	10%0=0	Si	"10 si es divisible entre 2"
2	7	3	NO	7%03:1	no	"7 no es divisible entre 3"
3	15	5	NO	15%05:0	si	"15 si es divisible entre 5"

Problema 1. Intervalo

Entrada . Un numero real x
Proceso : 1. Verificar si x es mayor o igual a 0 y si x es menor o igual a 10.
2. Si ambas condiciones se cumple, el número pertenece al intervalo, si alguna falla no pertenece
Salida Un mensaje indicando si el numero esta dentro del intervalo [0,10] o no.
Restricciones. El intervalo es cerrado, es decir, incluye los extremos 0 y 10. El número puede ser entero o decimal.

2. Tabla de valores

Objeto	Nombre	Valor	Tipo
Numero ingresado	x	5	real

3. Seudocodigo

Algoritmo Intervalo

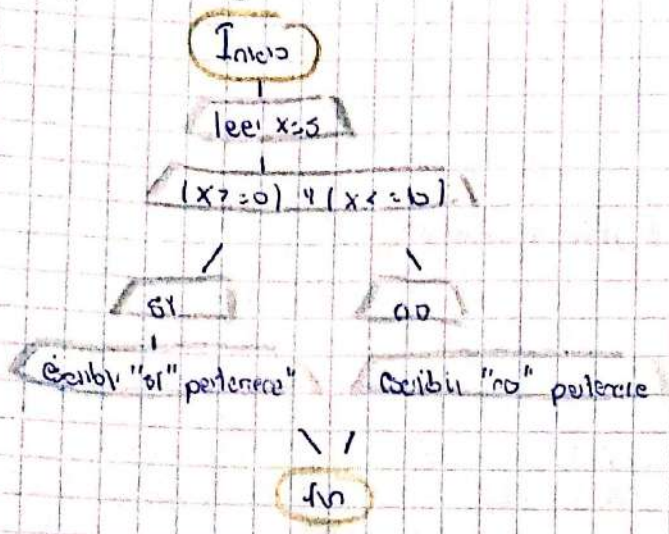
// 1. Declaración de variable
Definir x como real

// 2. Entrada de datos:
Escribir "Ingrese un numero"
→ leer x "5"

3. Proceso y salida
si (x>=0) y (x<=10) entonces
Escribir x: " si pertenece al intervalo [0,10]"
sino
escribir. x: "no pertenece al intervalo [0,10]"
fin si
fin Algoritmo.

Escaneado con CamScanner

4. Diagrama de flujo.



5. Pruebas de escritorio

Caso	x	x>=0	x<=10	(x>=0) y (x<=10)	Salida esperada
1	5	Verdadero	Verdadero	Verdadero	"5 si pertence al intervalo [0,10]"
2	-3	falso	verdadero	falso	"-3 no pertenece al intervalo [0;10]"
3	15	Verdadero	falso	falso	"15 no pertnec al intervalo [0;10]"

Problema 1.5 . Conversión de unidades de tiempo

Entrada: Un número entero c que representa la cantidad total de segundos

Proceso: 1. Calcular los días = c / 86400 (usando división entera)
2. Calcular el resto de segundos después de quitar los días : resto 1 = c % 86400
3. calcular las horas : horas = resto 1 / 3600
4. Calcular el resto de segundos después de quitar las horas resto 2 = resto 1 % 3600
5. Calcular los minutos : minutos = resto 2 / 60
6. Calcular los segundos finales : segundos = resto 2 % 60

Salida: Los valores de días, horas, minutos y segundos

Restricciones: El número c debe ser un entero no negativo. Si c es negativo el programa deberá mostrar un error

2. Tabla de valores.

Objeto	Nombre	Valor	Tipo
Segundos Totales	c	10000	entero
Días	días	0	entero
horas	horas	2	entero
minutos	minutos	46	entero
Segundos finales	segundos	40	entero
Resto Auxiliar 1	resto 1	10000	entero
Resto Auxiliar 2	resto 2	2800	entero.

Escaneado con CamScanner

3. Seudocodigo

Algoritmo Conversion Tiempo

// 1. Declaracion de variables
Definir c, dias, horas, minutos, segundos, resto 1, resto 2, como enteros

// 2. Entrada de datos
Escribir "Ingrese la cantidad de segundos"
Leer c "1000"

// 3. Proceso
// calculo de dias
// dias = 1000/86400 = 0
// resto 1 = 1000 % 86400 = 1000

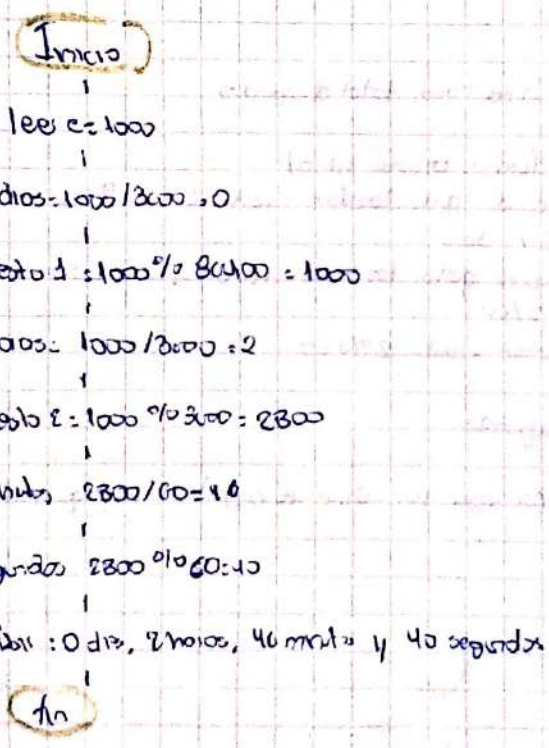
// calculo de horas
horas = 1000/3600 = 2
resto 2 = 1000 % 3600 = 2800

// calculo de minutos
minutos = 2800/60 = 46
segundos = 2800 % 60 = 40

// 4. Salida de resultados
Escribir c "equivale a:"
Escribir dias, "dias", horas, "horas", minutos, "minutos y" segundos

fin Algoritmo

4. Diagrama de flujo.



5. Pruebas de escritorio

Caso	c (entrada)	dias	resto 1	horas	resto 2	minutos	segundos	Salida
1	10000	0	10000	2	2800	46	40	"0 dias, 2h, 46min, 40seg"
2	3661	0	3661	1	61	1	1	"0 dias, 1h, 1min, 1seg"
3	86400	1	0	0	0	0	0	"1 dias, 0h, 0min, 0seg"

Escaneado con CamScanner

1.7. Raíz cuadrada entera

Entrada: Un número positivo N

Proceso: 1. Inicializar una variable i = 1
2. Mientras i² ≤ N, incrementar i en 1
3. Cuando el ciclo termine, el valor de i será el primer entero cuyo cuadrado supera a N por lo tanto, la raíz cuadrada entera por defecto es R = i - 1

Salida: El valor de R

Restricciones: N debe ser un valor positivo (N > 0) ya que N = 0, la raíz sería 0, pero el enunciado dice "positivo" así que asumimos N ≥ 1.

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Número ingresado	N	10	entero
Variable	i	4	entero
Raíz cuadrada	R	3	entero

3. Seudocódigo

```
Algoritmo Raiz_Cuadrada_Entera

// 1. Declaración de variables
Definir N, i, R como entero

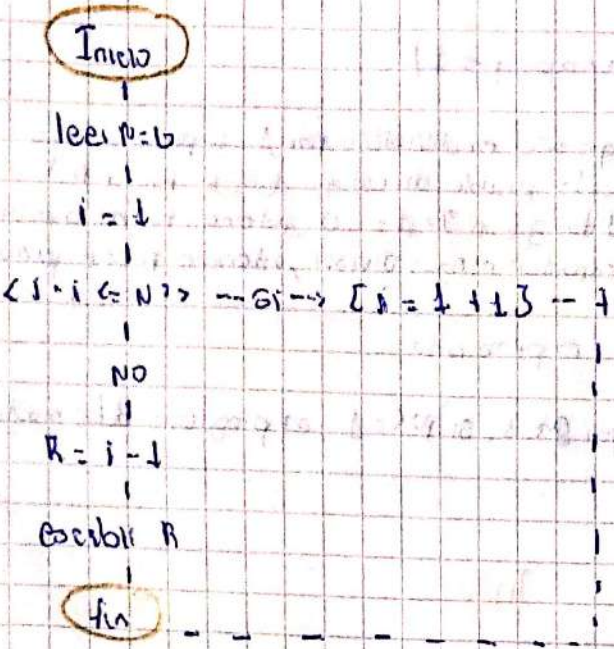
// 2. Entrada de datos
Escribir "Ingrese un número entero positivo"
Leer N

// 3. Proceso
i <- 1
Mientras (i * i) <= N Hacer
    i <- i + 1
Fin Mientras
R <- i - 1

// 4. Salida de resultados
Escribir "La raíz cuadrada entera por defecto de ", N, " es: ", R

Fin Algoritmo
```

4. Diagrama de flujo



5. Prueba de escritorio

caso	N	i inicial	¿1>1 <= N?	i	¿2>2 <= N?	i	¿3>3 <= N?	salida
1	0	1	si	2	si	3	si	3
2	16	1	si	2	si	3	si	4
3	1	1	si	2	no	-	-	1

Escaneado con CamScanner

1. S Número primo

- Entrada: Un número entero N (debe ser mayor que 1)
- Proceso: 1. Un número primo es aquel que solo es divisible por 1 y por sí mismo
  2. Para verificarlo, podemos recorrer los posibles divisores desde 2 hasta N-1.
  3. Si encontramos algún divisor d tal que N % d == 0 entonces N, no es primo
  4. Si terminamos el recorrido sin encontrar ningún divisor, entonces N sí es primo
- Salida: Un mensaje indicando si el número es primo o no
- Restricciones: N debe ser un entero mayor que 1. Si N <= 1, el programa debe mostrar error.

2. Tabla de objetos.

Objeto	Nombre	Valor	Tipo
Número ingresado	N	7	Entero
Divisor actual	divisor	2,3,4,5,6	Entero
Bandera de primo	esPrimo	Verdadero	Booleano

3. Seudocodigo

```
Algoritmo NumeroPrimo
    // 1. Declaracion de variables
    Definir N, divisor como Entero
    Definir esPrimo como Logico
    // 2. Entrada de datos
    Escribir "Ingrese un numero entero mayor que 1"
    Leer N
    // 3. Validacion
    Si N <= 1 entonces
        Escribir "Error: El numero debe ser mayor que 1"
    Sino
        // 4. Proceso
        esPrimo <- Verdadero
        divisor <- 2
        Mientras (divisor < N) Y (esPrimo = Verdadero) Hacer
            Si N % divisor = 0 entonces
                esPrimo <- Falso
            FinSi
```

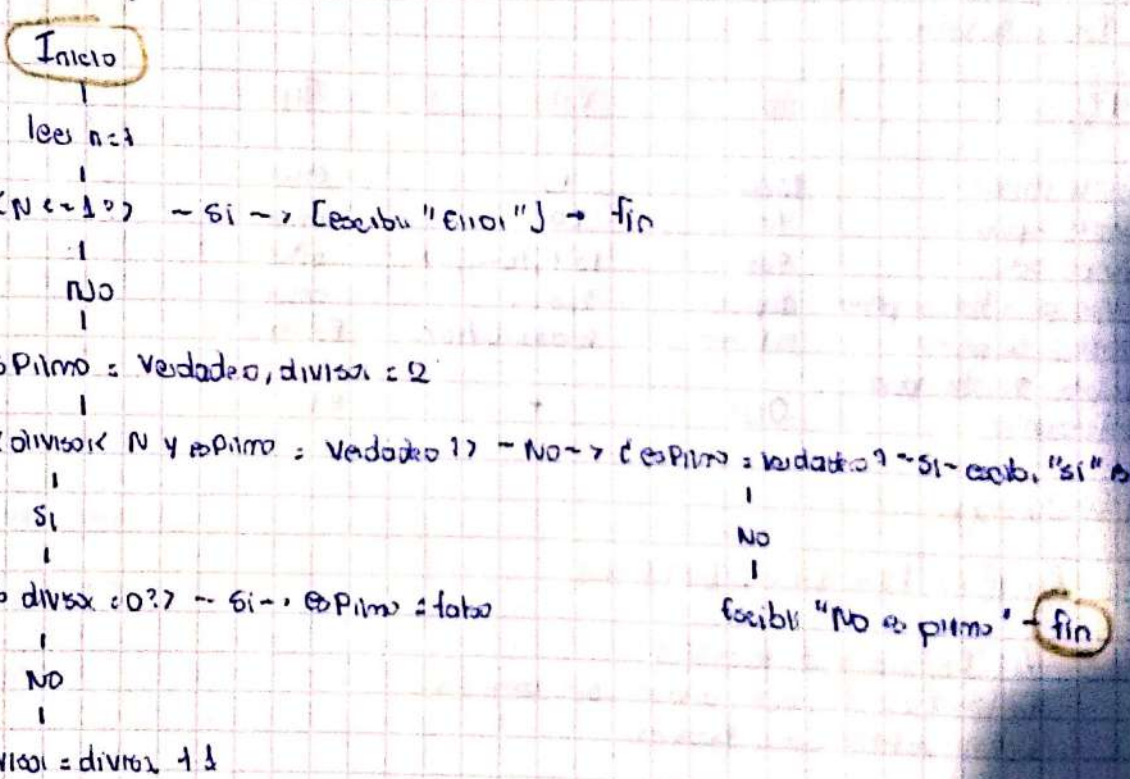
Escaneado con CamScanner

divisor ← divisor + 1
Fin Mientras

// 6. Salida de resultado

Si esPrimo = Verdadero entonces
Escribir N, "sí es número primo"
Sino
Escribir N, "No es número primo"
Fin Si
Fin Proceso
Fin del Algoritmo.

4. Diagrama de flujo



5. Prueba de escritorio

N	divisor	divisor < 7?	esPrimo?	N % divisor = 0?	Acción
1	2	Sí	Verdadero	No	divisor = 2
2	3	Sí	Verdadero	No	divisor = 3
3	4	Sí	Verdadero	No	divisor = 4

Escaneado con CamScanner

Problema L.9 Numeros primos comprendidos entre dos numeros dados

- Entrados. Dos numeros enteros que llamaremos inicio y fin
- Proceso. 1. Validar que inicio sea menor o igual que fin, sino, intercambiarlos.
  2. Recorrer todos los numeros desde inicio hasta fin
  3. Para cada numero, verificar si es primo
  4. Si es primo, mostrarlo por pantalla
  5.
- Salidas. La lista de numeros primos en el rango [inicio, fin]
- Restricciones. 1. Los numeros deben ser enteros
  2. si inicio > fin, se intercambian para que el rango sea valido

2. Tabla de Valores

Objeto	Nombre	Valor	Tipo
Limite inferior	inicio	10	entero
Limite superior	fin	20	entero
Numero actual	num	10, 11, 12, ..., 20	entero
Divisor para verificar primo	divisor	2, 3 ..	entero
Bandera de primo	esPrimo	verdadero / falso	Booleano
Variable auxiliar para intercambio	aux	-	entero

3. Seudocodigo

Algoritmo PrimosEntreDosNumeros

//1. Declaracion de variables
Definir inicio, fin, num, divisor, aux como entero
Definir esPrimo como Booleano

//2. Entrada de datos
Escribir "Ingrese el primer numero"
Leer inicio "10"
Escribir "Ingrese el segundo numero"
Leer fin "20"

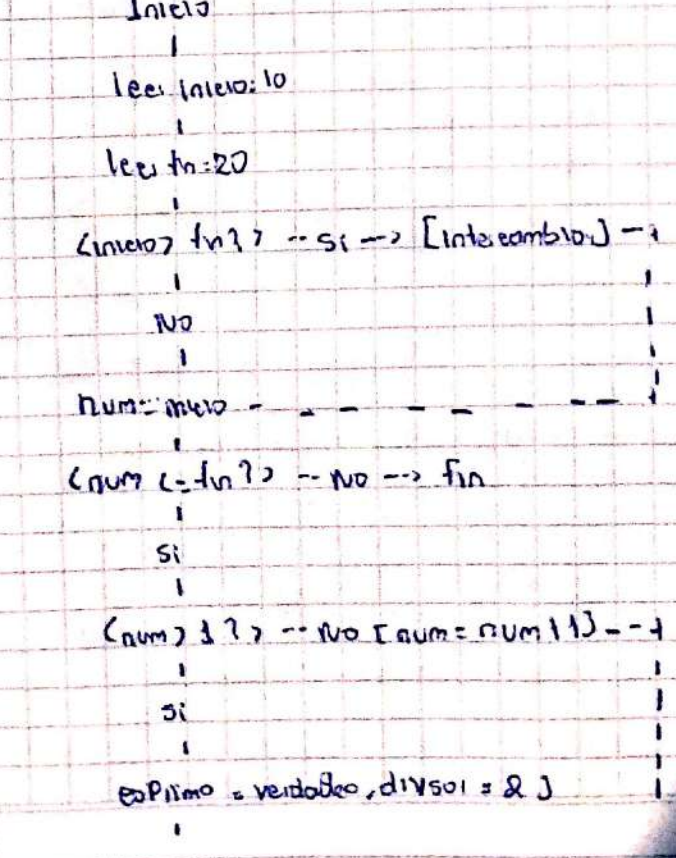
//3 validacion: asegurar que inicio <= fin
Si inicio > fin entonces
  aux <- inicio
  inicio <- fin
  fin <- aux
FinSi

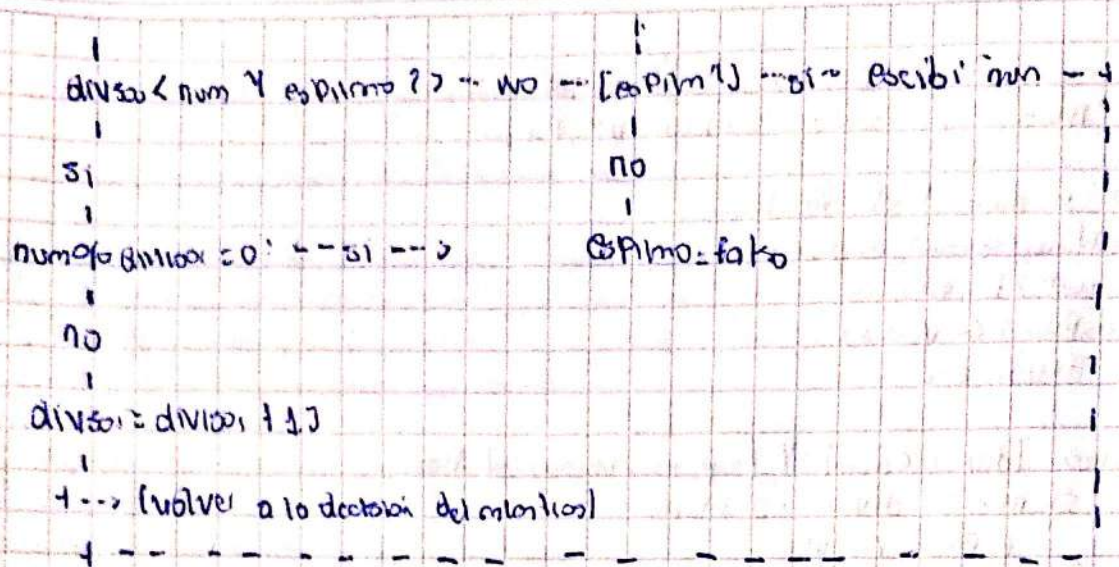
Escaneado con CamScanner

```
// 4. Proceso: leer el rango y verificar primos
Escribir "Numeros primos entre", inicio "y", fin

Para num <- inicio hasta fin Hacer
    // Verificar si num es primo
    Si num > 1 entonces
        esPrimo <- Verdadero
        divisor <- 2
        Mientras (divisor < num) Y (esPrimo = Verdadero) Hacer
            Si num % divisor = 0 entonces
                esPrimo <- falso
            finSi
            divisor <- divisor + 1
        finMientras
        Si esPrimo = Verdadero entonces
            Escribir num
        finSi
    finSi
finPara
finAlgoritmo.
```

4. Diagrama de flujo





[num = num + 1] → (vuelve a la decisión del Para

5. Pruebas de escritorio.

num	¿num > 1?	divisor	¿num%divisor = 0?	esPrimo	¿se muestra?
10	sí	2	10%2 = 0 → sí	falso	No
11	sí	2,3,4,5,6,7,8,9,10	ninguno divide a 11	Verdadero	Sí
12	sí	2	12%2 = 0 → sí	falso	No.

Escaneado con CamScanner

Problema 1. Lo factorial

- Entrada: Un número entero n (debe ser ≥ 0)
- Proceso: 1. Validar que n ≥ 0 si es negativo, mostrar error
  2. Inicializar un acumulador factorial = 1 (por 0! = 1 y es el elemento neutro de la multiplicación)
  3. Recorrer desde i = 1 hasta n, multiplicando el acumulado por i en cada paso
     factorial = factorial * i
  4. Al terminar factorial contiene el resultado de n!
- Salida: El valor de n!
- Restricciones: debe ser un entero no negativo

2. Tabla de objeto

Objeto	Nombre	Valor	Tipo
Número ingresado	n	5	entero
Contador del ciclo	i	1,2,3,4,5	entero
Acumulador del factorial	factorial	120	entero

3. Seudocódigo

Algoritmo factorial

```
//1. Declaración de variables
Definir n, i, factorial como Entero

//2. Entrada de datos
Escribir "Ingrese un número entero no negativo"
Leer n "si"

//3. Validación
Si n < 0 entonces
  Escribir "Error: El factorial no está definido para números negativos"
Sino

//4. Proceso
  factorial <- 1
  Para i <- 1 Hasta n Hacer
     factorial <- factorial * i
```

Escaneado con CamScanner

// Interacción 1: factorial = 1 * 1 = 1
// Interacción 2: factorial = 1 * 2 = 2
// Interacción 3: factorial = 2 * 3 = 6
// Interacción 4: factorial = 6 * 4 = 24
// Interacción 5: factorial = 24 * 5 = 120

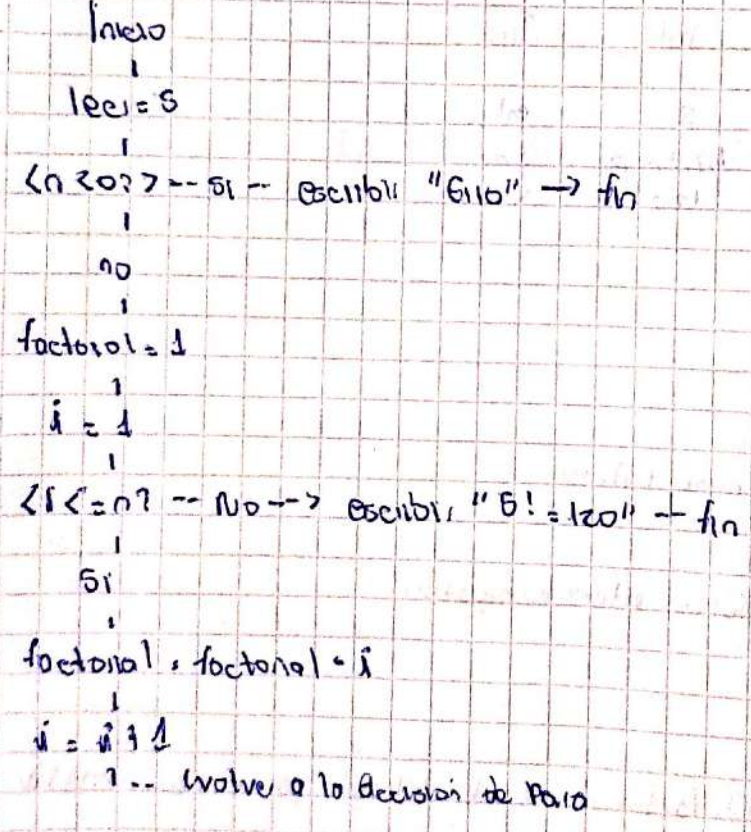
FinPara

//6. Salida de resultado
Escribir n, "!" factorial

FinSi

FinAlgoritmo.

4. Diagrama de flujo



5. Prueba de escritorio

Iteración	i	i<=5?	factorial
1	1	SI	1*1=1
2	2	SI	1*2=2
3	3	SI	2*3=6
4	4	NO	(fin)